Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Dönem projesi önerisi (proposal) formatı ve proje fikirleri.
- Birliktelik kuralları madenciliği (association rule mining) ve kullanım alanları: çapraz satış (cross-sell), üst satış (up-sell).
- Destek (support) ve güven (confidence) ölçütleri.
- Apriori algoritması: sık öğe kümeleri (frequent itemsets), aşağı kapalılık özelliği (downward closure property), aday üretimi (candidate generation), kural üretimi.
- Tablo verisinin işlem (transaction) formatına çevrilmesi.
- Sınıf birliktelik kuralları madenciliği (class association rule mining).
- Sıralı örüntü madenciliğine (sequential pattern mining) kısa giriş. Ayrıntısı gelecek hafta.

Hocanın Özellikle Vurguladığı Kısımlar

- Ders, kitabı bölüm bölüm takip ediyor. Her bölümden sonra ilgili bölümü, içindeki örnekleri ve bölüm sonu sorularını okumak öğrencinin sorumluluğunda.
- Vize ve final soruları kitaptaki ve slaytlardaki konu ve örneklerden geliyor. Başka yerden soru sorulmuyor.
- Proje önerisi 5. haftada isteniyor. Sonrasında yaklaşık 8 haftalık (2 aylık) çalışma süresi var.
- Bonus: Proje raporu IEEE iki kolonlu konferans formatında bir makaleye dönüştürülür, hoca onaylar ve bir konferansta kabul edilirse harf notu bir basamak yükseltilir. Not açıklandıktan sonra kabul gelse bile hoca bunu yapmaya çalışıyor.
- Apriori'nin maliyeti algoritmanın kendisinden değil, her seviyede veritabanının diskten baştan sona okunmasından geliyor.
- Hoca bir kümenin sık olması için iki şart saydı: destek eşiği aşılmalı ve tüm alt kümeleri sık olmalı. Destek şartı tek başına yeterlidir. Alt küme kontrolü, sayım yapmadan aday elemeye yarar.

Kısa Tekrar Notları

- İşlem (transaction): bir alışverişte birlikte alınan ürünler. I tüm öğeler, t bir işlem, T işlemler veritabanı.
- Kural: $X \to Y$. X ve Y, I'nın alt kümeleri ve $X \cap Y = \emptyset$.
- $\text{support}(X \to Y) = \dfrac{\text{count}(X \cup Y)}{n}$. n toplam işlem sayısı.
- $\text{confidence}(X \to Y) = \dfrac{\text{count}(X \cup Y)}{\text{count}(X)}$.
- Kullanıcı minimum support (minsup) ve minimum confidence (minconf) değerlerini verir.
- Algoritma tüm kuralları bulmalı, hiçbirini atlamamalı.
- Tek elemanlı kümeden kural çıkmaz. Kural için en az iki eleman gerekir.
- Apriori iki aşamalı:
  - Sık öğe kümelerini bul.
  - Bunlardan güveni minconf üstündeki kuralları üret.
- Aşağı kapalılık (Apriori özelliği): sık bir kümenin tüm alt kümeleri de sıktır.
- Aday üretimi iki adım: birleştirme (join) ve budama (prune).

- Kural üretimi: X sık küme, $A \subset X$ boş olmayan alt küme, $B = X - A$. $\text{conf}(A \to B) = \dfrac{\sup(X)}{\sup(A)}$.
- k seviye için veritabanı k kez okunur. Pratikte k için bir üst sınır konur (örneğin 10).
- Class association rule: kuralın sağ tarafında ürün yerine sınıf etiketi var.

Detaylı Açıklamalar

Dönem projesi önerisi

- Bir sayfalık öneride şunlar olacak:
  - Ne yapılacak ve neden yapılacak, motivasyon nedir?
  - Dersle ilgisi nedir, hangi konulara dayanıyor?
  - Veri seti nereden toplanacak? Hoca farklı kaynaklardan veri toplanmasını istiyor. Örnek kaynak: Papers with Code (makalelerin kodları ve veri setleri).
  - Yöntem nedir, nasıl yapılacak?
  - 8 haftalık iş planı: hangi hafta ne yapılacak?
- Örnek proje fikirleri:
  - Bir crawler yazmak.
  - PageRank ve diğer sıralama (ranking) mekanizmalarını bir graf verisi üzerinde uygulamak.
  - Topluluk bulma (community discovery) algoritmalarıyla bir graftaki alt ağları bulmak.
  - Birliktelik kurallarıyla bir veri setinde çapraz satış ve üst satış için birliktelikler bulmak.
- Öğrenciler kendi şirketlerindeki problemlerden de yola çıkabilir.
- Ders bir araştırma dersi gibi kurgulanmış. Proje; literatür taraması, doğruluk, gecikme süresi ve ölçeklenebilirlik açısından performans analizi, bir iki algoritmanın karşılaştırılması içerebilir.
- 5. haftada her öğrenci kamerasını açıp yaklaşık 5 dakikada önerisini anlatacak. Hoca eksikleri söyleyip onay verecek.
- Hoca, proje konusu için e-postayla randevu alınabileceğini söyledi. İlerleyen haftalarda salı derslerinden sonra ofis saati (office hour) yapacak.

Birliktelik kurallarının kullanımı

- Market sepeti analizi (market basket analysis): bir kasada tek seferde ödenen ürünler bir işlemdir.
- Çok sayıda insanın işlemlerinde birlikte geçen popüler birliktelikler bulunur. Örnek: süt ve ekmek.
- Çapraz satış: satılma potansiyeli yüksek bir ürünün yanında daha az popüler ama ilgili bir ürünü de satmak. E-ticaret sitelerinde ve fiziksel mağazalarda yoğun kullanılıyor.
  - Örnek: Merkez ofis tüm mağazalardan gelen işlemleri analiz eder. Birlikte satılan ürünler kasanın yanına konur veya kasada indirimle önerilir.
- Üst satış: bir ürünün veya hizmetin üst seviyesini satmak. Örnek: orta seviye bir üyeliği alan müşterilerin %90'ı sonradan üst seviyeye geçmişse, bu yükseltme yeni müşterilere önerilebilir.
- Problem ilk olarak 1993'te Agrawal ve arkadaşları tarafından tanımlandı. Bu ilk tanımda veriler kategorik kabul edildi, sayısal değil.
- Bu algoritmalar bugün Weka, Python kütüphaneleri, Spark ve Apache Flink gibi platformların makine öğrenmesi kütüphanelerinde var.
- Web madenciliğindeki kullanım: Her doküman bag of words olarak (bağlaç ve ünlem gibi anlamsız kelimeler atılarak) bir işlem gibi düşünülür. Sık geçen kelime grupları bulunur. Yüksek destekli bir birliktelik, doküman setinin hangi konulara (spor, politika vb.) yoğunlaştığını gösterir.

Support ve confidence

- Support, veri setinin kuralı ne kadar desteklediğini gösterir: işlemlerin yüzde kaçında X ve Y birlikte geçiyor?

- Confidence, kurala ne kadar güvenilebileceğini gösterir: X'i içeren işlemlerin yüzde kaçında Y de var?
- Bir öğrenci, ekmek almayıp süt alanların confidence hesabına girip girmediğini sordu.
  - Hoca: Girmez. "Ekmek → süt" kuralında şart ekmek almaktır. Payda yalnız ekmek içeren işlemlerdir.
  - Formüldeki birleşim işareti, hem ekmek hem süt içeren işlemleri sayar. Bunun ekmek içeren işlemler içindeki oranına bakılır.
- Örnek görev: Mağaza sahibi minsup %50 ve minconf %50 verir. Son 6 aylık verideki birlikte satılan ürünler ve bunlardan kurallar istenir.

Elle çözülen örnekler

- 5 işlemlik örnek, minsup %50:
  - Tekli sık öğeler: A (3), B (3), D (4), E (3) gibi.
  - İkili sık küme: {A, D} 5 işlemin 3'ünde geçiyor. Support %60.
  - $A \to D$: A içeren 3 işlemin 3'ünde D var. Confidence %100.
  - $D \to A$: D içeren 4 işlemin 3'ünde A var. Confidence %75.
  - İki kural da minconf %50 şartını sağlıyor.
- 7 işlemlik örnek, minsup %30, minconf %80:
  - En büyük sık küme: {chicken, clothes, milk}. 7 işlemin 3'ünde geçiyor ($3/7 \approx 43\%$, eşiğin üstünde).
  - Bu kümeden çıkan tüm kuralların support değeri aynı. Yalnız confidence ayrı ayrı hesaplanır.
  - Örnek: clothes → milk, chicken. Clothes içeren 3 işlemin 3'ünde ikisi de var. Confidence %100.

İşlem verisi

- En basit biçimde yalnız işlem ID'si ve ürün ID'leri var.
- Gerçek verilerde ürün miktarı ve fiyatı da olur. Algoritma için bunlar atılır, yalnız ürün ID'si kullanılır.

Apriori algoritması

- Girdi: veri seti, minsup ve minconf.
- Seviye seviye arama (level-wise search), aşağıdan yukarı (bottom-up) çalışır.
- Her iterasyonda veritabanı baştan sona bir kez taranır. Her satırda aday kümelerin sayaçları (count) artırılır.
- Adımlar:
  1. İlk tarama: tüm tekli öğelerin sayısı bulunur (C_1). Destek eşiğini geçenler F_1 olur.
  2. F_1 elemanları birleştirilerek tüm olası ikili adaylar (C_2) oluşturulur.
  3. Veritabanı yeniden taranır, ikililerin sayıları bulunur. Eşiği geçenler F_2 olur.
  4. F_{k-1}'den C_k üretilir. Alt kümelerinden biri F_{k-1}'de olmayan adaylar sayılmadan atılır.
  5. Yeni aday kalmayana kadar devam edilir.
- Örnek (4 işlem, minsup %50, yani en az 2 işlem):
  - $F_1 = \{1\}, \{2\}, \{3\}, \{5\}$.
  - C_2: {1,2}, {1,3}, {1,5}, {2,3}, {2,5}, {3,5}.
  - Sayımdan sonra F_2: {1,3}, {2,3}, {2,5}, {3,5}.
  - Üçlü adaylardan yalnız {2,3,5} budamadan geçer. Alt kümeleri {2,3}, {2,5}, {3,5} sık.
  - Son taramada {2,3,5} 2 işlemde bulunur. Sık küme olarak kabul edilir.

Aday üretimi (candidate-gen)

- Birleştirme adımı (join step): $(k-1)$ elemanlı sık kümeler birleştirilerek k elemanlı adaylar üretilir.
- Budama adımı (prune step): bir alt kümesi önceki seviyenin sık listesinde olmayan adaylar elenir.

- Örnek: sık üçlüler {1,2,3}, {1,2,4}, {1,3,4}, {1,3,5}, {2,3,4} gibi.
  - Birleştirmede {1,2,3,4}, {1,2,3,5} ve {1,3,4,5} gibi dörtlüler oluşur.
  - {1,2,3,4}: tüm üçlü alt kümeleri sık. Kalır.
  - {1,3,4,5}: {3,4,5} sık değil. Elenir.
  - {1,2,3,5}: {2,3,5} sık değil. Elenir.
- Budamadan geçen adayın sayımı bir sonraki veritabanı taramasında yapılır.

Kural üretimi

- Apriori'nin bulduğu sık kümeler henüz kural değildir. Ayrı bir aşama gerekir.
- Sık küme X için tüm $A \to (X - A)$ kuralları yazılır.
- X sık olduğu için bütün bu kurallar minsup şartını zaten sağlar. Yalnız confidence hesaplanır.
- Örnek: $X = \{2, 3, 4\}$, support %50. Alt kümelerin support değerleri verilmiş.
  - $\{2, 3\} \to \{4\}$: $50/50 = \%100$.
  - $\{2, 4\} \to \{3\}$: $50/50 = \%100$.
  - $\{3, 4\} \to \{2\}$: $50/75 \approx \%67$.
  - Tekli sol taraflar için de ($\{2\} \to \{3, 4\}$ vb.) aynı hesap yapılır.
  - minconf %70 olsaydı yalnız ilk iki kural yazılırdı.

Maliyet

- k büyüklüğündeki en büyük kümeyi bulmak için veritabanı k kez sıfırdan okunur.
- Milyonlarca satırlık işlem tablolarında disk okuma (I/O) bellekten okumaya göre çok pahalı. Bu yüzden k büyüdükçe algoritma çok yavaşlar.

Tablo verisini işlem formatına çevirme

- Apriori, her satırı bir işlem olan veriyi bekler.
- Tablo biçimindeki veri için her hücre "(öznitelik adı, değer)" çiftine çevrilir. Örnek: (Attribute1, a), (Attribute2, b), (Attribute3, d). Böylece her satır bir işlem olur.

Class association rule mining

- Aynı Apriori mantığı kullanılır. Fark: kuralın sağ tarafında ürün yerine sınıf etiketi var.
- Örnek veri: 7 doküman, her birinin kelimeleri ve etiketi (education veya sport).
  - $\{\text{student}, \text{school}\} \to$ education: sol tarafı içeren 2 dokümanın 2'si education. Confidence $2/2$, support $2/7$.
  - $\{\text{game}\} \to$ sport: game içeren 3 dokümanın 2'si sport. Confidence $2/3$.
- Bir öğrenci bunun TF-IDF'e benzeyip benzemediğini sordu.
  - Hoca: TF-IDF, bir kelimenin bir doküman setindeki önemini ölçer. Terim sıklığı (term frequency) dokümandaki geçiş sayısına, ters doküman sıklığı (inverse document frequency) kelimenin kaç dokümanda geçtiğine bakar. Her dokümanda geçen kelimenin ayırt edici gücü azdır.
  - Class association rule ise birlikte geçen kelimeleri ve bunların konularını bulur.
- Bir öğrenci etiketlerin veri setinde olup olmadığını sordu.
  - Hoca: Etiket mutlaka verilmeli. Ya açıkça (explicit) ya dolaylı (implicit) gelir. Örnek: "100 dokümanın 30'u education, 70'i sport" denir. Her dokümandan bag of words çıkarılır, yanına etiket eklenir ve işlem verisi oluşturulur.

Sıralı örüntü madenciliği (giriş)

- Aksiyonların hangi sırayla alınacağını bulan algoritmalar.
- Müşteri kaybı (churn) analizi ve müşteri elde tutma (retention) modellerinde kullanılıyor. Daha önce sistemi terk eden müşterilere uygulanan başarılı aksiyon sıraları öğrenilebilir.
- Gelecek hafta işlenecek.

Ders sonrası notlar

- Hoca 5 işlemlik örnekte $A \to D$ için "desteği %100, güvenilirliği %100" dedi. Doğrusu: support %60, confidence %100.
- Class association rule örneğinde hoca önce $\{\text{game}\} \to \text{sport}$ için support $2/7$ dedi, sonra bunu yanlış bulup $3/7$ olarak düzeltti. İlk söylediği büyük olasılıkla doğrudur (kitaptaki tanım altyazıdan doğrulanmıyor): sınıf kuralının support değeri, hem X'i içeren hem de etiketi y olan işlem sayısının toplam işlem sayısına oranıdır. Bu örnekte $2/7$. Yalnız sol tarafı içeren işlemlerin oranı ($3/7$) kuralın support değeri değildir.
- Hoca sık küme kontrolünü "sayılarının minimum support'un üzerinde olması" diye anlattı. Doğrusu "eşit veya üzerinde". Örnekte de 4 işlemden 2'sinde geçenler sık sayıldı.
- Hoca birleştirme adımını anlatırken "1-3 ile 2-5'i merge ediyor" dedi. Standart Apriori'de birleştirme, ilk $k-2$ elemanı aynı olan kümeler arasında yapılır. Hangi adayların çıktığı slayttaki tabloya bağlı, altyazıdan tam anlaşılmıyor.
- Problem 1993'te tanımlandı. Apriori adıyla bilinen algoritma Agrawal ve Srikant'ın 1994 tarihli çalışmasıdır.
- Altyazıdaki bozuk adlar: "Appure", "Appiori", "Apiary", "Epriori" doğrusu Apriori; "Secure Pattern Mining" doğrusu Sequential Pattern Mining; "Candidate Jam/Chain" doğrusu candidate-gen; "Absel" doğrusu up-sell; "Veka" doğrusu Weka.
- Hoca algoritmanın en fazla elemanlı sık kümeyi bulup kuralları ondan çıkardığını söyledi. Kurallar büyük olasılıkla bütün sık kümelerden üretilir. Yalnız en büyük kümeden yazılırsa daha küçük bir sık kümenin kuralları kaçar.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.